

0. OBTENCIÓN DE ARCHIVOS

Obtengo los archivos con los que trabajaré de la siguiente web:

https://insideairbnb.com/fr/get-the-data/?utm_source=chatgpt.com

De ahí descargo dos archivos: calendar y listings. Ambos están en formato .csv, pero transformo el de calendar en .xlsx para así poder usar en Python dos archivos de diferente formato y cumplir el requisito del proyecto. Esto no altera los datos, solo el contenedor.

1. CARGA DE DATOS

1.1. Importación de librerías

Primero importo las librerías de numpy y pandas:

```
import numpy as np
```

```
import pandas as pd
```

1.2. Lectura de .csv y .xlsx

Cargo los dos archivos (.csv y .xlsx):

```
listado = pd.read_csv("C:/Users/sarav/Documents/Cursos/Data analytics The Power/9. Proyecto Final/listingsMADRID.csv")
```

```
calendario = pd.read_excel("C:/Users/sarav/Documents/Cursos/Data analytics The Power/9. Proyecto Final/calendarMADRID.xlsx")
```

2. EXPLORACIÓN INICIAL DE DATOS

2.1. Número de filas y columnas, tipos de datos, primeras filas...

Primero se quiere hacer una revisión inicial de ambos conjuntos de datos para revisar:

- Número de filas y columnas
- Tipos de datos (dtypes) → La columna Price esta como texto
- Primeras filas (head)

LISTADO

Códigos:

```
Listado.info()
```

```
Listado.head()
```

```
Listado.shape
```

```
Listado["id"].nunique()
```

Con esto se concluye que el conjunto de datos listado tiene 25.000 filas con valores únicos y 79 columnas.

CALENDARIO

```
calendario.info()
```

```
calendario.head()
```

```
calendario.shape
```

```
calendario["listing_id"].nunique()
```

Se puede ver que el conjunto de datos tiene 1.048.575 filas, las cuales pertenecen a 2.873 ids únicos.

También se puede ver que el campo común entre ambos dataframes es el id de listado y el listing_id de calendario.

2.2. Nulos

Ahora, se procede a revisar los valores nulos de ambos conjuntos de datos.

LISTADO

```
listado.isnull()
```

```
listado.isnull().sum()
```

```
id                0
listing_url       0
scrape_id         0
last_scraped      0
source            0
...
calculated_host_listings_count      0
calculated_host_listings_count_entire_homes  0
calculated_host_listings_count_private_rooms  0
calculated_host_listings_count_shared_rooms  0
reviews_per_month                    5147
Length: 79, dtype: int64
```

Al ser muchas columnas no me salen los nulos de todas, así que lo expreso de otra manera para que me saque las columnas que tienen más nulos:

```
nulos = listado.isnull().sum().sort_values(ascending=False)
```

```
nulos.head(20)
```

```
calendar_updated      25000
host_neighbourhood     17078
license                15812
neighbourhood          14414
neighborhood_overview  14414
host_about             12402
host_location          8150
estimated_revenue_l365d  6047
price                  6047
bathrooms              6040
beds                   6035
review_scores_value     5151
review_scores_location  5150
review_scores_checkin   5148
review_scores_cleanliness  5148
review_scores_communication  5147
reviews_per_month       5147
review_scores_accuracy  5147
first_review           5147
review_scores_rating    5147
dtype: int64
```

CALENDARIO

`calendario.isnull()`

`calendario.isnull().sum()`

```
listing_id      0
date            0
available       0
price          1048575
adjusted_price  1048575
minimum_nights  0
maximum_nights  0
dtype: int64
```

Aquí se ven valores nulos solo en las columnas de Precio y de Precio ajustado, pero son iguales a la totalidad del número de filas.

2.3. Duplicados

`listado.duplicated().sum()`

`calendario.duplicated().sum()`

Para ambos el resultado es 0 asique no hay filas duplicadas en ninguno de los dos dataframes.

3. TRANSFORMACION Y LIMPIEZA PROFUNDA

3.1. Identificación de tipos de datos por columnas

Primero nos vamos a centrar en el Dataframe **LISTADO**

Vamos a ver de que tipo son cada una de las columnas:

```
listado.dtypes.value_counts()
```

```
str          36  
float64      23  
int64        20  
Name: count, dtype: int64
```

¿Cuáles de las columnas string podrían ser realmente numéricas o fechas? Mostramos las 35 columnas de texto:

```
listado.select_dtypes(include="string").columns.tolist()
```

```
['listing_url',  
'last_scraped',  
'source',  
'name',  
'description',  
'neighborhood_overview',  
'picture_url',  
'host_url',  
'host_name',  
'host_since',  
'host_location',  
'host_about',  
'host_response_time',  
'host_response_rate',  
'host_acceptance_rate',  
'host_is_superhost',  
'host_thumbnail_url',  
'host_picture_url',  
'host_neighbourhood',  
'host_verifications',  
'host_has_profile_pic',  
'host_identity_verified',  
'neighbourhood',  
'neighbourhood_cleansed',  
'neighbourhood_group_cleansed',  
...  
'calendar_last_scraped',  
'first_review',  
'last_review',  
'license',  
'instant_bookable']
```

Hay varias de esa lista que de momento llaman la atención como `last_scraped` o `first_review` porque parecen fechas y ahora mismo están como texto. Vamos a comprobarlo:

```
listado[
```

```
[  
    "last_scraped",  
    "host_since",  
    "calendar_last_scraped",  
    "first_review",  
    "last_review"  
]  
].head()
```

	last_scraped	host_since	calendar_last_scraped	first_review	last_review
0	2025-09-15	2010-02-21	2025-09-15	2014-10-10	2018-07-15
1	2025-09-15	2010-05-24	2025-09-15	2010-07-06	2025-08-27
2	2025-09-15	2010-05-26	2025-09-15	2015-05-12	2017-05-30
3	2025-09-15	2010-05-24	2025-09-15	2010-11-01	2025-09-11
4	2025-09-15	2010-11-29	2025-09-15	2018-05-10	2025-09-05

Efectivamente, hay que cambiar esos campos a formato fecha.

Ahora voy a comprobar el formato de la columna PRICE:

```
listado["price"].dtype
```

está en formato string asique también tendremos que cambiarlo a float.

Voy a comprobar tambien los valores de porcentajes y booleanos que actualmente están como string:

```
listado[
```

```
    [  
        "host_response_rate",  
        "host_acceptance_rate",  
        "host_is_superhost",  
        "host_has_profile_pic",
```

```

        "host_identity_verified",
        "instant_bookable"
    ]
].head()

```

	host_response_rate	host_acceptance_rate	host_is_superhost	host_has_profile_pic	host_identity_verified	instant_bookable
0	100%	NaN	f	t	t	f
1	100%	100%	f	t	t	t
2	NaN	NaN	f	t	f	f
3	100%	100%	f	t	t	t
4	100%	99%	f	t	t	f

Tras estos análisis iniciales, sacamos las siguientes conclusiones de variables a cambiar:

VARIABLE	FORMATO ACTUAL	FORMATO DESEABLE
Last_scraped	string	datetime
Host_since	string	datetime
Calendar_last_scraped	string	datetime
First_review	string	datetime
Last_review	string	datetime
price	string	float
Host_response_rate	string	float
Host_acceptance_rate	string	float
Host_is_superhost	string	Bool
Host_has_profile_pic	string	Bool
Host_identity_verified	string	Bool
Instant_bookable	string	Bool

3.2. Conversión de variables

a) Convertir las columnas de string a fecha

Primero se comenzará convirtiendo las columnas que tenían formato string y deberían tener formato de fecha:

- Last_scraped
- Host_since
- Calendar_last_scraped
- First_review
- Last_review

```

columnas_fecha = [
    "last_scraped",

```

```
"host_since",  
"calendar_last_scraped",  
"first_review",  
"last_review"  
]
```

```
for col in columnas_fecha:  
    listado[col] = pd.to_datetime(listado[col])
```

```
listado[columnas_fecha].dtypes
```

```
last_scraped      datetime64[us]  
host_since        datetime64[us]  
calendar_last_scraped  datetime64[us]  
first_review      datetime64[us]  
last_review       datetime64[us]  
dtype: object
```

b) Convertir la columna de Price a formato numérico

```
listado["price"] = (  
    listado["price"]  
    .str.replace("$", "", regex=False)  
    .str.replace(",", "", regex=False)  
    .astype(float)  
)
```

```
listado["price"].describe()
```



```

count      18953.000000
mean       156.691447
std        492.255350
min         8.000000
25%        70.000000
50%       110.000000
75%       164.000000
max       25654.000000
Name: price, dtype: float64

```

Se ha convertido correctamente y además queda demostrado porque se pueden sacar parámetros propios de un campo numérico como media, mediana, etc...

c) Convertir las columnas de string a porcentajes

```

listado["host_response_rate"] = (
    listado["host_response_rate"]
    .str.replace("%", "", regex=False)
    .astype("float")
)

```

```

listado["host_acceptance_rate"] = (
    listado["host_acceptance_rate"]
    .str.replace("%", "", regex=False)
    .astype("float")
)

```

```

listado[
    ["host_response_rate", "host_acceptance_rate"]
].describe()

```

	host_response_rate	host_acceptance_rate
count	20115.000000	20996.000000
mean	90.130748	84.170699
std	22.359971	27.306349
min	0.000000	0.000000
25%	94.000000	84.000000
50%	100.000000	98.000000
75%	100.000000	100.000000
max	100.000000	100.000000

d) Convertir las columnas de string a booleanos

```
columnas_bool = [
    "host_is_superhost",
    "host_has_profile_pic",
    "host_identity_verified",
    "instant_bookable"
]

for col in columnas_bool:
    listado[col] = listado[col].map({
        "t": True,
        "f": False
    })
```

`listado[columnas_bool].dtypes`

```
host_is_superhost      object
host_has_profile_pic   object
host_identity_verified  object
instant_bookable        bool
dtype: object
```

No todas las columnas se han convertido a formato bool. Esto quiere decir que esas columnas tienen más valores aparte de t y f. Lo comprobamos:

```

for col in [
    "host_is_superhost",
    "host_has_profile_pic",
    "host_identity_verified"
]:
    print(f"\n{col}")
    print(listado[col].unique())

```

```

host_is_superhost
[False True nan]

host_has_profile_pic
[True False nan]

host_identity_verified
[True False nan]

```

Como vemos, los valores posibles de esas columnas son: True, False y nan.

Vamos a cambiarlas al tipo de booleano que acepta el campo nan.

```

columnas_bool = [
    "host_is_superhost",
    "host_has_profile_pic",
    "host_identity_verified"
]

for col in columnas_bool:
    listado[col] = listado[col].astype("boolean")

listado[columnas_bool].dtypes

```

```

host_is_superhost      boolean
host_has_profile_pic   boolean
host_identity_verified  boolean
dtype: object

```

Este es el resumen de los cambios que se han hecho:

COLUMNA	TIPO ORIGINAL	TIPO FINAL
Price	String	Float
Host_since	String	Datetime
First_review	String	Datetime
Host_response_rate	String(%)	Float
Host_acceptance_rate	String(%)	Float
Host_is_superhost	Str	boolean
Host_has_profile_pic	Str	Boolean
Host_identity_verified	Str	Boolean
Instant_bookable	str	bool

3.3. Tratamiento de nulos de gran porcentaje

Antes de comenzar con el tratamiento de los nulos, queremos ver las 15 columnas con mayor porcentaje de nulos:

```
nulos_pct = (listado.isnull().sum() / len(listado) * 100).sort_values(ascending=False)
```

```
nulos_pct.head(15)
```

```
calendar_updated      100.000
host_neighbourhood     68.312
license               63.248
neighbourhood         57.656
neighborhood_overview 57.656
host_about            49.608
host_location         32.600
estimated_revenue_l365d 24.188
price                 24.188
bathrooms             24.160
beds                  24.140
review_scores_value    20.604
review_scores_location 20.600
review_scores_checkin  20.592
review_scores_cleanliness 20.592
dtype: float64
```

Vamos a empezar a analizar el campo de calendar_updated:

```
listado["calendar_updated"].head()
```

```
0    NaN
1    NaN
2    NaN
3    NaN
4    NaN
Name: calendar_updated, dtype: float64
```

Como vemos tiene un 100% de valores nulos asique no aporta nada de información relevante y, por tanto, se puede eliminar.

Vamos a analizar el segundo campo con mayores nulos:

```
listado["host_neighbourhood"].value_counts().head(10)
```

```
host_neighbourhood
Sol                858
Malasaña          627
Embajadores       581
La Latina         520
Cortes            450
Justicia          437
Palacio           330
San Blas          183
Argüelles         175
Ciudad Lineal     158
Name: count, dtype: int64
```

Tiene información bastante relevante por lo que es más difícil decidir si eliminar este campo o no. Se ve que puede tener información parecida a la variable `neighbourhood_cleansed` asique se procede a compararlas:

```
listado["neighbourhood_cleansed"].isnull().sum()
```

No tiene nulos y como la información es parecida, se decide eliminar la columna de `host_neighbourhood` y dejar la de `neighbourhood_cleansed`.

Ahora procedemos a analizar la tercera columna con más nulos → `license`

```
listado["license"].head(10)
```

```

0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
5  VT-7440
6      NaN
7      NaN
8  VT-6993
9      NaN
Name: license, dtype: str

```

License es la columna que contiene los números de identificación de licencia turística. Tiene un porcentaje de nulos bastante elevado (63.2%). Es un dato que no nos va a dar mucha información en el análisis, por lo que se decide eliminar.

Tras revisar la descripción de las variables `neighbourhood` con `neighbourhood_cleansed`, se observa que contienen información similar, por lo que se procede a compararlas.

```
listado[
```

```
    ["neighbourhood", "neighbourhood_cleansed"]
```

```
].head(10)
```

	neighbourhood	neighbourhood_cleansed
0	Madrid, Spain	Cármenes
1	NaN	Sol
2	NaN	Embajadores
3	NaN	Universidad
4	Madrid, Spain	Justicia
5	Madrid, Community of Madrid, Spain	Universidad
6	Madrid, Spain	Universidad
7	Madrid, Spain	Universidad
8	NaN	Embajadores
9	NaN	Recoletos

La primera columna da datos de ciudad/comunidad autónoma y la segunda del barrio. Al ser todo datos de Airbnb en Madrid, no creo que la primera columna aporte mucha información, pero vamos a ver sus valores primeros para analizarlo:

```
listado["neighbourhood"].value_counts(dropna=False).head(10)
```

```

neighbourhood
NaN 14414
Madrid, Comunidad de Madrid, Spain 9576
Madrid, Community of Madrid, Spain 688
Madrid, Spain 247
Madrid, Co, Spain 19
Madrid Center, Madrid, Spain 10
Madrid , Madrid, Spain 8
MADRID, Spain 7
Chueca, Madrid, Spain 5
Pozuelo de Alarcón, Comunidad de Madrid, Spain 2
Name: count, dtype: int64

```

Decido eliminar la columna de neighbourhood porque casi todos los valores son de Madrid ciudad, cosa que tiene sentido porque los datos son de ahí. No aporta demasiado.

Procedemos a eliminar la quinta columna con más nulos (57.7%).

```
listado["neighborhood_overview"].dropna().head(3)
```

```

0    We live in a leafy neighbourhood with plenty o...
4    DISTRICT WITH VERY GOOD VIBES IN THE MIDDLE OF...
5    The location of this apartment is ideal for wa...
Name: neighborhood_overview, dtype: str

```

Como vemos, los valores son descripciones de texto largas sobre el barrio, por lo que eliminaremos esa columna porque no aporta valor para un análisis.

Haremos lo mismo con la siguiente columna de host_about (49.6% de nulos)

```
listado["host_about"].dropna().head(3)
```

```

0    EN-ES-FR\r\nEN\r\nHi everybody: I'm Abdel. I'm...
1    Apartasol offers a network of several spacious...
2    Estoy empezando en Airbnb y deseo que mis hués...
Name: host_about, dtype: str

```

Ocurre lo mismo asique también eliminaremos esa columna.

Ahora vamos a ver si hay algunas columnas cuyo resultado sean enlaces URL o imágenes ya que tampoco nos aportarían información útil y podríamos eliminarlas:

```
[
```

```
col for col in listado.columns
    if "url" in col.lower() or "picture" in col.lower()
```

```
]
```

```
['listing_url',
 'picture_url',
 'host_url',
 'host_thumbnail_url',
 'host_picture_url']
```

Se obtienen esas columnas asique se eliminarán también.

3.4. Eliminación de columnas irrelevantes

El resumen de las columnas que se ha decidido eliminar es el siguiente:

COLUMNA	ESTRATEGIA	MOTIVO
Calendar_updated	Eliminar	100% nulos
host_neighbourhood	Eliminar	68% nulos y redundante
License	Eliminar	Identificador con 63% de nulos que no aportada utilidad
Neighbourhood	Eliminar	Muchos nulos y aportaban información redundante de la ciudad
neighborhood_overview	Eliminar	Campo texto extenso que no aportaba información
Host_about	Eliminar	Campo texto extenso que no aportaba información
Listing_url	Eliminar	Link
Picture_url	Eliminar	link
Host_url	Eliminar	link
Host_thumbnail_url	Eliminar	link
Host_picture_url	Eliminar	link

Se proceden a eliminar con los siguientes comandos:

```
listado = listado.drop(columns=["calendar_updated"])
```

```
listado = listado.drop(columns=["host_neighbourhood"])
```

```
listado = listado.drop(columns=["license"])
```

```
listado = listado.drop(columns=["neighbourhood"])
```



```

listado = listado.drop(columns=["neighborhood_overview"])
listado = listado.drop(columns=["host_about"])
columnas_url = [
    "listing_url",
    "picture_url",
    "host_url",
    "host_thumbnail_url",
    "host_picture_url"
]
listado = listado.drop(columns=columnas_url)

```

3.5. Tratamiento de nulos de menor porcentaje

Comprobamos el estado de los valores nulos de nuevo:

```

nulos_pct = (
    listado.isnull().sum()
    / len(listado)
    * 100
).sort_values(ascending=False)
nulos_pct.head(10)

```

```

host_location      32.600
price              24.188
estimated_revenue_l365d  24.188
bathrooms          24.160
beds               24.140
review_scores_value  20.604
review_scores_location  20.600
review_scores_cleanliness  20.592
review_scores_checkin  20.592
review_scores_rating  20.588
dtype: float64

```

Las columnas que quedan pueden tener valores útiles para el análisis, así que hay que pensarlo mejor antes de eliminarlas o no.

Vamos a ir analizándolas en orden de porcentaje de nulos:

```
listado["host_location"].value_counts(dropna=False).head(15)
```

```
host_location
Madrid, Spain      14582
NaN                8150
Barcelona, Spain   678
Spain              153
London, United Kingdom  94
Las Rozas, Spain   45
Málaga, Spain      33
Paris, France       29
Alicante, Spain     28
Mexico City, Mexico  26
Buenos Aires, Argentina  24
Community of Madrid, Spain  23
New York, NY        22
Sant Joan d'Alacant, Spain  21
Zarzalejo, Spain    20
Name: count, dtype: int64
```

De momento se mantiene porque puede tener información interesante sobre el origen de los propietarios.

Vamos a evaluar otras columnas con nulos, como bathroom:

```
listado["bathrooms"].describe()
```

```
count    18960.000000
mean      1.310021
std       0.704779
min       0.000000
25%       1.000000
50%       1.000000
75%       1.500000
max       19.000000
Name: bathrooms, dtype: float64
```

```
listado["bathrooms"].head(10)
```

```
0    NaN
1    1.0
2    NaN
3    1.0
4    1.5
5    NaN
6    1.0
7    1.0
8    1.0
9    NaN
Name: bathrooms, dtype: float64
```

Con estos resultados, se decidirá más adelante qué hacer con ellos.

Ahora vamos a analizar la columna beds:

```
listado["beds"].describe()
```

```
count    18965.000000
mean      1.986291
std       1.523673
min       0.000000
25%       1.000000
50%       2.000000
75%       2.000000
max       40.000000
Name: beds, dtype: float64
```

`listado["beds"].head(10)`

```
0    NaN
1    2.0
2    NaN
3    3.0
4    2.0
5    NaN
6    1.0
7    2.0
8    5.0
9    NaN
Name: beds, dtype: float64
```

Con estos resultados, se decidirá más adelante qué hacer con ellos.

Vamos a analizar la columna Price y estimated_revenue_l365d:

```
listado[
    ["price", "estimated_revenue_l365d"]
].isnull().sum()
```

```
price                6047
estimated_revenue_l365d  6047
dtype: int64
```

Mismo numero de nulos en ambas columnas.

```
(
    listado["price"].isnull()
    ==
    listado["estimated_revenue_l365d"].isnull()
)
```

```
).mean()
```

```
np.float64(1.0)
```

Con estos resultados, se decidirá más adelante qué hacer con ellos.

Analizamos las columnas de las valoraciones:

```
listado.loc[
    listado["review_scores_rating"].isnull(),
    "number_of_reviews"
].describe()
```

```
count      5147.0
mean         0.0
std          0.0
min          0.0
25%          0.0
50%          0.0
75%          0.0
max          0.0
Name: number_of_reviews, dtype: float64
```

Se comprueba que los valores nulos de las variables de valoración coinciden con alojamientos que no han recibido aún ninguna reseña (`number_of_reviews=0`). Por tanto, los nulos de las valoraciones no son un error de calidad del dato. El alojamiento aún no tiene reseñas. Por ello no voy a hacer imputaciones en ninguna de las variables relacionadas con `review_scores` porque me estaría inventando un dato que no existe.

Para los de menor %:

COLUMNA	ESTRATEGIA
Host_location	Mantener de momento
Bathrooms	Pendiente de decidir
Beds	Pendiente de decidir
Review_score	Mantener los nulos porque si no estaríamos cambiando la realidad
Price	Pendiente de decidir

3.6. Tratamiento de nulos del dataframe CALENDAR

```
calendario.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1048575 entries, 0 to 1048574
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   listing_id            1048575 non-null  int64
1   date                  1048575 non-null  datetime64[us]
2   available              1048575 non-null  str
3   price                  0 non-null        float64
4   adjusted_price         0 non-null        float64
5   minimum_nights         1048575 non-null  int64
6   maximum_nights         1048575 non-null  int64
dtypes: datetime64[us](1), float64(2), int64(3), str(1)
memory usage: 56.0 MB
```

Las columnas de price y adjusted_price tienen un 100% de nulos. En principio podríamos eliminarlos porque no parece que sirvan.

```
calendario = calendario.drop(columns=["price"])
```

```
calendario = calendario.drop(columns=["adjusted_price"])
```

Vamos a comprobar si la columna available tiene valores booleanos:

```
calendario["available"].value_counts(dropna=False)
```

```
available
f    741670
t    306905
Name: count, dtype: int64
```

Efectivamente se puede cambiar a formato booleano:

```
calendario["available"] = (
    calendario["available"]
    .map({"t": True, "f": False})
    .astype("boolean")
)
```

```
calendario["available"].dtype
```

```
BooleanDtype
```

Comprobamos que hayamos hecho bien las transformaciones:

```
calendario.isnull().sum()
```

```
listing_id      0
date            0
available       0
minimum_nights  0
maximum_nights  0
dtype: int64
```

3.7. Unión de fuentes – Merge

Hacer primero el merge antes de seguir limpiando lo que queda de listings

```
df = calendario.merge(
    listado,
    left_on="listing_id",
    right_on="id",
    how="left"
)
```

```
dataset.shape
```

tiene 1.048.575 filas y 73 columnas.

Antes de seguir, voy a guardar una copia porque este es el dataset principal del análisis.

```
dataset.to_csv(
    "airbnb_madrid_merged.csv",
    index=False
```

)

Procedo a hacer una auditoria final de los nulos del dataset final:

(

```
dataset.isnull().sum()
```

```
/ len(dataset)
```

```
* 100
```

).sort_values(ascending=False).head(15)

```
beds                44.590516
estimated_revenue_1365d  44.590516
bathrooms           44.590516
price               44.590516
host_response_rate   41.214029
host_response_time   41.214029
host_acceptance_rate  37.802732
bedrooms            21.651289
host_location        14.306559
review_scores_value   13.505949
review_scores_location 13.471139
reviews_per_month    13.436330
review_scores_accuracy 13.436330
review_scores_rating  13.436330
review_scores_cleanliness 13.436330
dtype: float64
```

Review_scores → no hago nada con ese campo porque son nulos legítimos como ya vimos

Hos_location → no es demasiado un 14% y la variable es interesante como para borrarla.

Vamos a ver las demás:

```
dataset[
```

```
[
```

```
    "price",
```

```
    "bathrooms",
```

```
    "beds"
```

```
]
```

```
J.isnull().corr()
```

	price	bathrooms	beds
price	1.0	1.0	1.0
bathrooms	1.0	1.0	1.0
beds	1.0	1.0	1.0

Esto significa que los nulos aparecen en las mismas filas. Es decir, cuando falta el precio también faltan las camas y los baños.

Si al eliminar esas filas de datos nulos, siguen quedando más de 50.000 filas de datos, creo que lo más adecuado es borrarlas.

Borro y compruebo el número final:

```
dataset_eda = dataset.dropna(  
    subset=["price", "bathrooms", "beds"]  
)
```

```
dataset_eda.shape
```

Finalmente, quedan 581.010 filas y 73 columnas así que como se cumplen los requisitos y es una muestra grande para sacar conclusiones, se dejan así los datos habiendo borrado esas filas nulas.

Con este dataset final doy por terminada la fase de limpieza. He eliminado columnas inútiles, corregido formatos, analizado y justificado los nulos y aun así queda un archivo grande para analizar.

4. ANÁLISIS DESCRIPTIVO

Ahora con los datos tratados, se procede a hacer diferentes comparaciones y análisis que nos ayuden a sacar conclusiones:

4.1. Análisis 1: Precio

¿Cómo se distribuyen los precios de los alojamientos en Madrid?

Para ello se hace un análisis descriptivo de la variable precio.

```
dataset_eda["price"].describe()
✓ 0.3s
```

count	581010.000000
mean	171.225616
std	656.793483
min	15.000000
25%	79.000000
50%	119.000000
75%	176.000000
max	25654.000000
Name: price, dtype: float64	

Primeras conclusiones:

1. La media (171€) es bastante mayor que la mediana (119€), por lo que existen alojamientos muy caros que empujan la media hacia arriba.
2. Hay mucha dispersión (desviación típica 656€) comparada con una media de 171€
3. Hay valores extremos, tanto en mínimo como en máximo

Vamos a ver los alojamientos más caros:

```
dataset_eda["price"].sort_values(ascending=False).head(20)
✓ 0.1s
```

109177	25654.0
109178	25654.0
109179	25654.0
109180	25654.0
109149	25654.0
109152	25654.0
109193	25654.0
109194	25654.0
109195	25654.0
109196	25654.0
109165	25654.0
109166	25654.0
109151	25654.0
109136	25654.0
109137	25654.0
109138	25654.0
109139	25654.0
109140	25654.0
109141	25654.0
109142	25654.0
Name: price, dtype: float64	

Los 20 registros más caros tienen exactamente el mismo precio. No parece algo natural. Necesitamos saber si esos 20 registros pertenecen a un único alojamiento repetido muchos días o a varios alojamientos distintos:

```
dataset_eda.loc[
    dataset_eda["price"] == dataset_eda["price"].max(),
    "id"
].nunique()
✓ 0.0s
1
```

Al ser el resultado 1, confirmamos que es un único alojamiento con estos precios extremadamente altos. Aparece repetido porque el calendario genera una fila por fecha.

Esto nos da la idea de que para los análisis de alojamientos (precio, barrio, habitaciones, etc) es mejor usar un dataset sin duplicados por alojamiento.

Por ejemplo:

```
alojamientos = dataset_eda.drop_duplicates(subset="id")
✓ 0.4s

alojamientos.shape
✓ 0.0s
(1592, 73)
```

Dan 1592 alojamientos.

Son relativamente pocos alojamientos ya que en listados se había observado que había 25.000 alojamientos. Solo 1.592 alojamientos tienen datos completos, es decir, han sobrevivido a la limpieza. Al pasar el archivo calendar de csv a xlsx pueden haberse perdido filas porque Excel tiene un número máximo de datos.

Aun así, se continua con esta muestra ya que se puede sacar mucha información y cumple los requisitos del proyecto.

El archivo Calendar fue convertido a formato XLSX para cumplir el requisito del proyecto de trabajar con diferentes formatos de datos. Debido a la limitación de Excel de 1.048.576 filas por hoja, el conjunto de datos quedó truncado durante la conversión. No obstante, tras la fase de limpieza y transformación se dispone de una muestra de 581.010 registros, tamaño suficiente para realizar análisis descriptivos y obtener conclusiones representativas.

4.2. Análisis 2: Barrios más caros

¿Cuáles son los barrios de Madrid con los alojamientos más caros?

```
alojamientos = dataset_eda.drop_duplicates(subset="id")

alojamientos.groupby(
    "neighbourhood_cleaned"
)[["price"]].agg(["count", "mean"]).sort_values(
    "mean",
    ascending=False
)
```

2] ✓ 0.5s

	count	mean
neighbourhood_cleaned		
Casa de Campo	5	512.200000
Recoletos	18	359.888889
Sol	153	352.869281
Rios Rosas	10	272.800000
Trafalgar	20	256.750000
...	...	...
Orcasitas	1	40.000000
Canillejas	1	38.000000
San Pascual	2	34.000000
Zofío	1	24.000000
Abrantes	1	20.000000

107 rows x 2 columns

Hay que tener cuidado con los barrios con muy pocos alojamientos como puede ser el caso de Casa de Campo, Orcasitas o Zofío.

Para obtener conclusiones, me centraré en los barrios con al menos 10 alojamientos:

```
barrios_precio = (
    alojamientos.groupby("neighbourhood_cleaned")["price"]
    .agg(["count", "mean"])
)

barrios_precio = barrios_precio[barrios_precio["count"] >= 10]

barrios_precio.sort_values("mean", ascending=False)
```

✓ 0.0s

	count	mean
neighbourhood_cleaned		
Recoletos	18	359.888889
Sol	153	352.869281
Rios Rosas	10	272.800000
Trafalgar	20	256.750000
Castellana	16	225.187500
Cortes	83	224.662651
Argüelles	17	203.764706
Justicia	86	196.534884
Comillas	12	173.166667
Acacias	14	172.071429
Universidad	190	159.636842
Palacio	145	158.227586
Palos de Moguer	58	152.241379
Ibiza	18	150.555556
Embajadores	246	149.947154
Lista	11	140.272727
Bellas Vistas	13	140.230769
Pacífico	16	139.687500
Goya	24	136.916667
Guindalera	12	133.166667
Adelfas	13	130.230769
Gaztambide	15	105.866667
Almenara	10	100.000000

Se observa que los precios más elevados se concentran en zonas céntricas y de alto poder adquisitivo de Madrid.

Se puede concluir con esto que la localización es uno de los factores que más influyen en el precio de los alojamientos turísticos de Madrid. Los barrios céntricos presentan precios medios significativamente superiores a los barrios periféricos.

4.3. Análisis 3: Superhost vs No Superhost

¿Los Superhosts cobran más que los anfitriones normales?

```
alojamientos.groupby(  
    "host_is_superhost"  
)[  
    "price"  
].agg(["count", "mean"])
```

0.0s

	count	mean
host_is_superhost		
False	930	154.187097
True	613	196.168026

Los alojamientos gestionados por Superhosts presentan un precio medio superior: 196€ vs 154€.

La diferencia es aproximadamente 42€ por noche.

Los alojamientos gestionados por anfitriones con la categoría Superhost presentan un precio medio de 196 €, frente a los 154 € de los anfitriones sin dicha distinción. Esto sugiere que la experiencia, reputación y calidad percibida de los Superhosts les permite fijar precios más elevados.

4.4. Análisis 4: Habitaciones y precio

¿Hay una relación entre el número de habitaciones y el precio del alojamiento?

```
alojamientos.groupby(  
    "bedrooms"  
)[  
    "price"  
].agg(["count", "mean"]).sort_values("mean")
```

0.0s

	count	mean
bedrooms		
1.0	898	108.042316
0.0	127	120.228346
2.0	382	258.693717
3.0	125	265.736000
4.0	40	368.400000
5.0	9	420.777778
7.0	5	678.400000
9.0	1	1000.000000
6.0	5	1065.400000

Conclusión: Existe una relación positiva entre el número de dormitorios y el precio del alojamiento.

En general, los alojamientos más grandes presentan precios más elevados.

La tendencia general muestra un incremento del precio conforme aumenta el número de dormitorios. Sin embargo, las categorías con más de cinco dormitorios presentan un número reducido de observaciones, por lo que sus resultados deben interpretarse con cautela.

4.5. Análisis 5: Tipo de alojamiento

¿Qué tipo de alojamiento es más caro?

```
alojamientos.groupby(  
    "property_type"  
)[["price"]].agg(["count", "mean"]).sort_values(  
    "mean",  
    ascending=False  
).head(15)
```

✓ 0.0s

	count	mean
property_type		
Entire serviced apartment	62	299.258065
Room in boutique hotel	7	263.714286
Entire home	13	241.923077
Room in hotel	2	218.000000
Entire rental unit	998	201.391784
Room in hostel	13	163.153846
Private room in guest suite	4	154.500000
Entire condo	53	153.698113
Entire hostel	2	153.500000
Entire guest suite	3	135.666667
Private room in serviced apartment	1	135.000000
Entire loft	82	122.109756
Room in aparthotel	8	118.500000
Entire place	3	111.000000
Private room in tiny home	1	94.000000

Centrándome en las categorías con suficiente volumen de alojamientos, podemos observar que los alojamientos enteros (entire serviced apartment, entire rental unit, entire condo y entire loft) presentan precios significativamente superiores a las habitaciones individuales.

Tiene sentido debido a la mayor privacidad, capacidad, etc.

4.6. Análisis 6: Valoraciones

¿Los alojamientos mejor valorados son más caros?

```
alojamientos["grupo_valoracion"] = pd.cut(
    alojamientos["review_scores_rating"],
    bins=[0, 4, 4.5, 4.8, 5],
    labels=["Baja", "Media", "Alta", "Excelente"]
)
```

```
alojamientos.groupby("grupo_valoracion")["price"].agg(["count", "mean"])
```

	count	mean
Baja	23	126.434783
Media	214	136.135514
Alta	747	153.285141
Excelente	582	205.256014

He creado unas categorías de valoraciones y de ahí analizar el precio. Se observa una ligera tendencia al aumento del precio en los alojamientos con mejores valoraciones. Sin embargo, las diferencias encontradas son reducidas, por lo que no puede afirmarse que exista una relación fuerte entre la valoración de los

huéspedes y el precio del alojamiento.

4.7. Conclusiones finales

1. Los precios de los alojamientos presentan una elevada dispersión, con una media de 171 € y una mediana de 119 €, lo que indica la existencia de alojamientos de lujo que elevan el precio medio.
2. Los barrios céntricos como Sol o Recoletos presentan precios medios significativamente superiores al resto de zonas de Madrid.
3. Los Superhosts gestionan alojamientos con precios medios más elevados que los anfitriones estándar.
4. Existe una relación positiva entre el número de dormitorios y el precio del alojamiento.
5. Los alojamientos completos presentan precios superiores a habitaciones privadas o alojamientos compartidos.
6. No se observan diferencias relevantes entre las valoraciones de los huéspedes y el precio de los alojamientos.

Después de estos pasos y estas conclusiones, guardo el archivo final para poder sacar las visualizaciones en Power BI:

```
dataset_eda.to_csv(
    "airbnb_madrid_limpio.csv",
    index=False
)
```

5. VISUALIZACION Y DASHBOARD

5.1. Importar archivo

Abro Power BI y desde el inicio hago estos pasos:

Obtener datos → Texto o CSV → elijo el archivo → delimitado coma → cargar

5.2. Visualizaciones

Página 1 - Resumen general

En esta página el objetivo es ver cómo es el mercado Airbnb en Madrid.

Para ello vamos a mostrar unos KPIs en forma de tarjeta:

- Nº alojamientos → `count(id)`
- Precio medio → `avg(price)`
- Valoración media → `avg(review_scores_rating)`

Me he dado cuenta que el campo de `review_scores_rating` tenía valores entre 300 y 500 así que la valoración está en formato de cien. He creado una columna que divida esta columna entre 100 llamada Valoración real.

Kpi con la tarjeta de `avg(Valoracion_Real)`

- % superhosts → se creará una medida:
% Superhosts =
`DIVIDE(CALCULATE(COUNTROWS(Alojamientos), Alojamientos[host_is_superhost] = TRUE()),COUNTROWS(Alojamientos))`

Además, se van a insertar varios gráficos:

Gráfico 1: Distribución de tipos de alojamiento

Gráfico de barras horizontales con los siguientes datos:

- Eje Y: `property_type`
- Eje X: Recuento de id

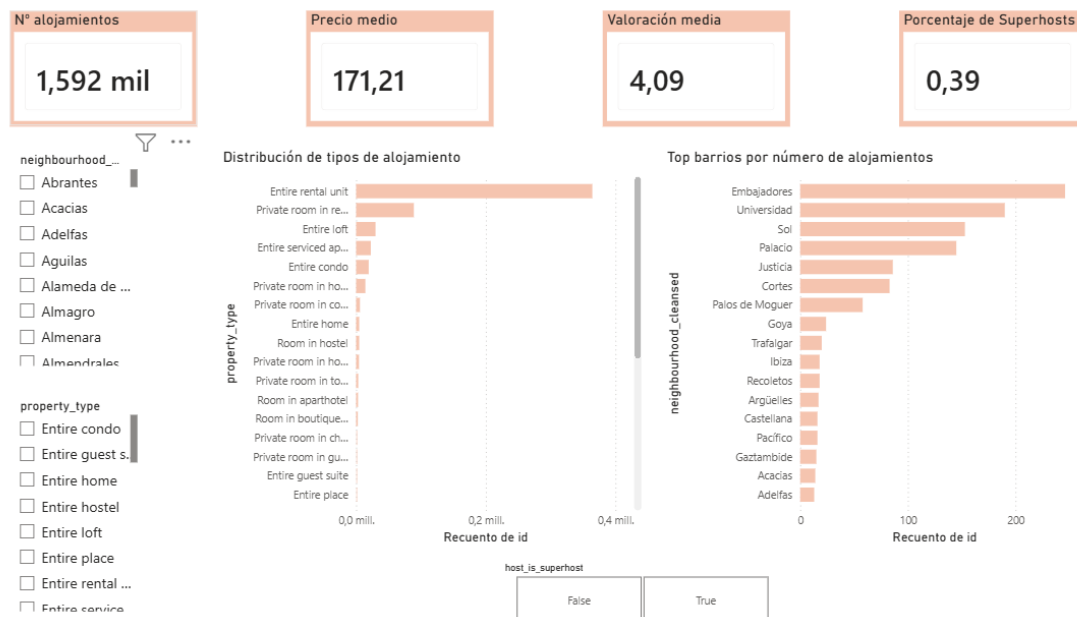
En base al resultado, el alojamiento que predomina es “Entire rental unit”

Gráfico 2: Top barrios por número de alojamientos

Gráfico de barras horizontales con los siguientes datos:

- Eje Y: neighbourhood_cleansed
- Eje X: Recuento de id

De aquí podemos concluir que los 5 barrios con mayor numero de alojamientos son: Embajadores, Universidad, Palacio, Sol y Justicia.



Página 2 – Barrios

El objetivo de esta pagina es concluir donde están los alojamientos y cuales son los barrios más caros.

Para ello se mostrarán los siguientes gráficos:

Gráfico 1: Precio medio por barrio

Gráfico de barras horizontales con los siguientes datos:

- Eje Y: neighbourhood_cleansed
- Eje X: Promedio de Price

Le aplico un filtro de Top N = 10 para que me muestre los 10 valores mayores.

De aquí podemos concluir que los 10 barrios de mayor precio son:

1. Casa de Campo
2. Recoletos
3. Sol

4. Rios Rosas
5. Trafalgar
6. Los Rosales
7. Orcasur
8. Castellana
9. Cortes
10. Almagro

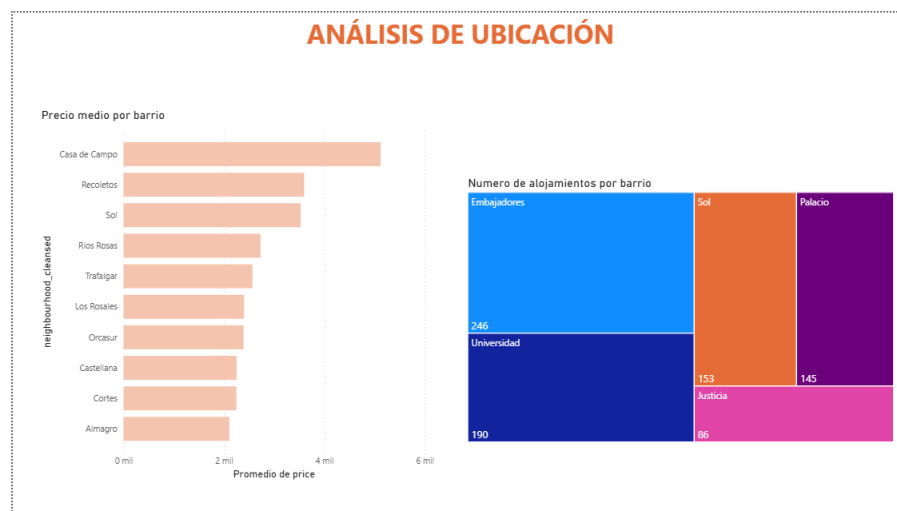
Gráfico 2: Numero de alojamientos por barrio

Elijo un gráfico de treemap para mostrar el top 5 barrios por número de alojamiento:

- Categoría: neighbourhood_cleansed
- Valores: recuento de id

Le aplico un filtro de top 5 valores y el resultado es que los 5 barrios con más alojamientos son:

1. Embajadores
2. Universidad
3. Sol
4. Palacio
5. Justicia



Con estos resultados podemos decir que el barrio más caro para alojarse es Casa de Campo y el barrio donde encontramos más alojamientos es Embajadores.

Página 3 – Características del alojamiento

El objetivo de esta página es concluir si el anfitrión y las características del alojamiento influyen en el precio del mismo.

Gráfico 1: Superhost vs No Superhost

Elijo un gráfico de barras verticales con los siguientes datos:

- Eje X: host_is_superhost
- Eje Y: Promedio de price

Gráfico 2: Precio medio según número de dormitorios

Elijo un gráfico de líneas con los siguientes datos:

- Eje X: habitaciones
- Eje Y: promedio de precio

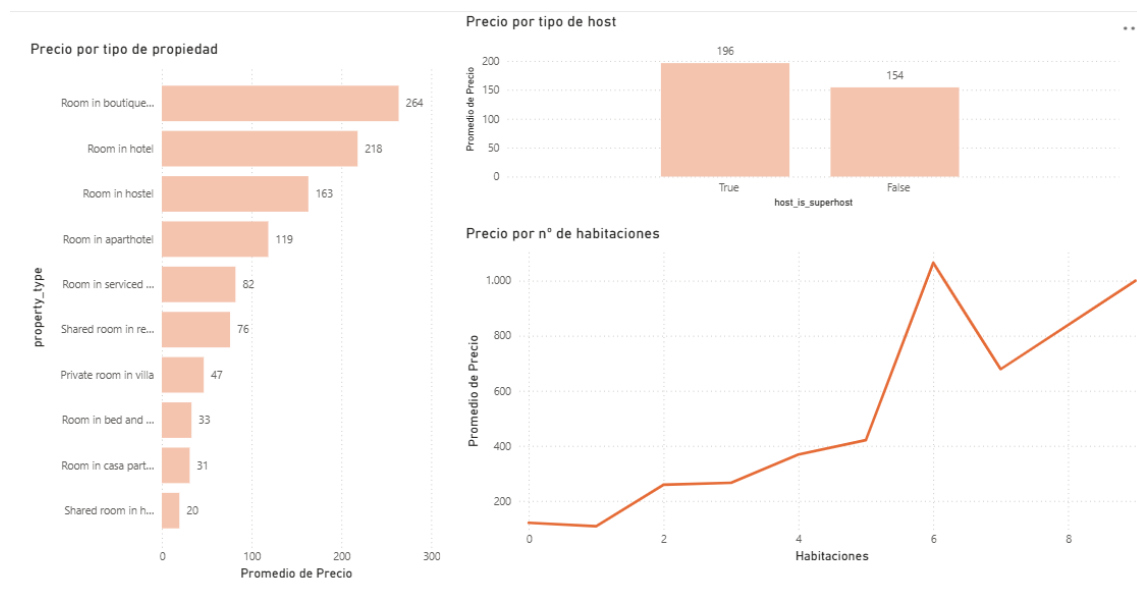
Gráfico 3: Precio medio según tipo de alojamiento

Hago un grafico de barras horizontales con los siguientes datos:

- Eje X: promedio de precio
- Eje Y: property_type

Aplicamos un top 10 al haber muchos tipos de alojamientos.

CARACTERISTICAS DEL ALOJAMIENTO



Con estos resultados si podemos concluir que ser Superhost influye en poder poner un precio mayor del alojamiento. Tambien tener más habitaciones y tener un apartamento turístico completo.

Los alojamientos con 6 o más habitaciones tenían muy pocos registros y por ello la línea presenta saltos.

Página 4 – Valoraciones

El objetivo de esta página es determinar qué relación tienen las valoraciones con los alojamientos.

Gráfico 1: Valoración media por barrio

- Eje Y: neighbourhood_cleansed
- Eje X: promedio de Valoración_Real

Aplicamos un top 10 de resultados de mayor a menor.

Gráfico 2: Valoración media por tipo de alojamiento

- Eje Y: property_type
- Eje X: promedio de Valoración_Real

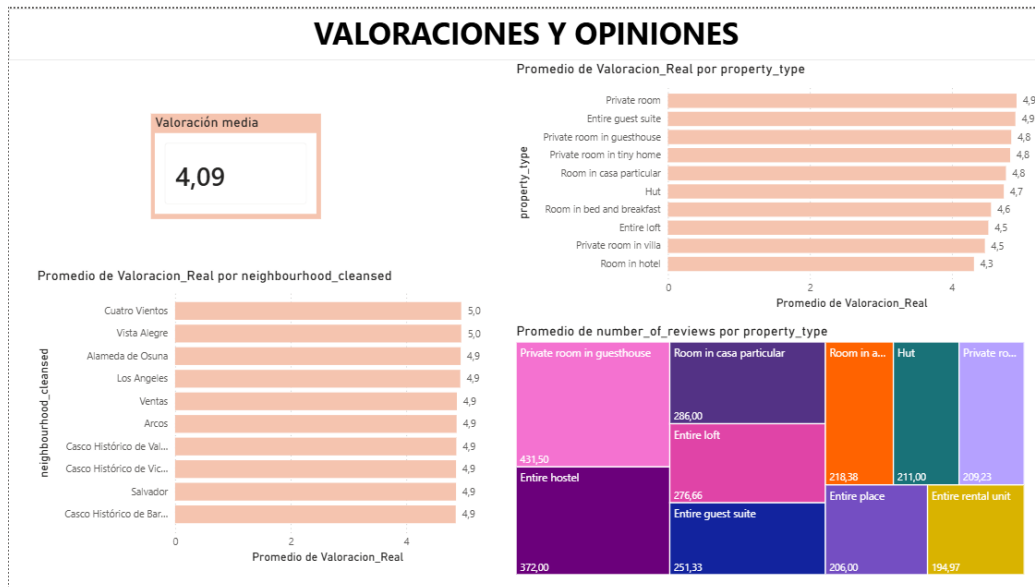
Aplicamos un top 10 de resultados de mayor a menor.

Gráfico 3: Número de reseñas por tipo de alojamiento

Hago un tree map con los siguientes datos:

- Categoría: property_type
- Valores: promedio de number_of_reviews

Además, he incluido la tarjeta con el KPI de valoración media:



Con este resultado se puede concluir lo siguiente:

- Los barrios con mejores valoraciones son Cuatro Vientos y Vista Alegre aunque no coinciden con ser los barrios con mayor número de alojamientos.
- Los alojamientos con mejor valoración son los que corresponden a habitaciones privadas y alojamientos completos privados.
- También estos son de los que más reseñas reciben junto a las habitaciones en casas particulares.

En conjunto, los resultados muestran que el precio de los alojamientos turísticos en Madrid está más condicionado por factores estructurales del alojamiento (ubicación, tamaño, tipo de propiedad y categoría de Superhost) que por las valoraciones recibidas por los huéspedes. La localización y las características del inmueble son los elementos que presentan una mayor influencia sobre el precio final.